

01- Mathematics & Statistics

LEVEL 2 — INTERVIEWER TEST | 15 Intermediate to Semi-Advanced Questions

Format: Each question includes a conversational student-style answer with code snippets and real-world reasoning.

Section A — Descriptive Questions (4 Questions)

Q1

Explain the difference between Population Variance (σ^2) and Sample Variance (s^2). Why is Bessel's correction necessary, and when would you use each in a data analysis project?

Answer

Great question — this trips up a lot of people! The core difference is about what we're measuring and whether we have the full picture.

Population Variance (σ^2) is used when you have data for every single member of the group you're interested in. You divide by N (the full count). Sample Variance (s^2) is used when you only have a subset of the population, which is almost always the case in real data work. You divide by $(n - 1)$ instead of n .

Why $n - 1$? Here's the intuition: a sample tends to cluster closer to its own mean than the full population does. So if you just divide by n , you'd consistently underestimate the true spread. Subtracting 1 gives back that 'lost' degree of freedom and removes the bias. This is called Bessel's correction.

```
import numpy as np
```

```
data = [4, 7, 13, 2, 1]
```

```
# Population variance (you surveyed EVERY employee in a small team)
```

```
pop_var = np.var(data, ddof=0) # ddof=0 → divide by N
```

```
# Sample variance (you surveyed 5 out of 500 employees)
```

```
samp_var = np.var(data, ddof=1) # ddof=1 → divide by N-1 (Bessel's correction)
```

```
print(f'Population Variance: {pop_var:.2f}')
```

```
print(f'Sample Variance: {samp_var:.2f}')
```

Real-world rule of thumb: In almost every data science project you use sample variance ($ddof=1$) because you're working with collected data, not a census. Population variance is rare — maybe a full database of all transactions in one month, or a fixed controlled experiment where you have all subjects.



Section B — Scenario-Based Descriptive Questions (4 Questions)

Q5

SCENARIO: You are an HR analyst at a tech company with 50 employees. 48 employees earn between ₹40,000–₹80,000/month, but the CEO earns ₹30,00,000/month and the CTO earns ₹20,00,000/month. A reporter asks you: 'What is the average salary at your company?' — How would you respond, and which measure(s) would you use and why?

✓ Answer

This is a classic case where the word 'average' is ambiguous and the choice of measure has ethical and practical implications.

If I just report the mean, it would be something like ₹1,38,000/month (heavily inflated by the CEO/CTO). That's technically correct but misleading — it implies the 'typical' employee earns well above what they actually do. A candidate offered a job would feel deceived.

I'd report the median salary. With 50 data points, the median would likely fall around ₹55,000–65,000/month, which actually reflects where most employees sit. I'd say: 'The median salary is ₹X — this represents what a typical employee earns. Our mean salary appears higher due to senior executive compensation.'

```
import numpy as np
```

```
# Simulated salaries
```

```
salaries = [40000 + i * 1000 for i in range(48)] + [2000000, 3000000] # 48 staff + 2 execs
```

```
print(f'Mean Salary: ₹{np.mean(salaries):,.0f}')
```

```
print(f'Median Salary: ₹{np.median(salaries):,.0f}')
```

```
print(f'CV (spread): {(np.std(salaries)/np.mean(salaries)*100):.1f}%') # Very high CV = unequal distribution
```

Additionally, I might include the IQR (Q1–Q3 range) to show where the 'middle 50%' of employees fall — this gives the full picture without a single number being misleading. If the reporter insists on 'average', I'd use the trimmed mean (dropping the top 5%) as a compromise that gives a mean-like figure without the outlier distortion.

Key takeaway: The right measure depends on your audience and the story the data tells. In salary reporting, median is the industry standard precisely for this reason.

02- Excel Google Sheets

Level 2: Interviewer Test | 15 Semi-Advanced Questions

All answers follow the conversational student format — thoughtful, practical, with code examples.

Descriptive Questions (4)

Q1. [Descriptive]

Explain the three major advantages of INDEX-MATCH over VLOOKUP. In which professional situation would you absolutely mandate INDEX-MATCH over VLOOKUP?

➡ Answer:

So when I think about INDEX-MATCH vs VLOOKUP, I always frame it around three core advantages that matter in real work:

1. Left Lookup Capability

The biggest one for me is that VLOOKUP forces the lookup column to be the leftmost column — you can only look right. INDEX-MATCH breaks that completely. With `=INDEX(A:A, MATCH(D2, E:E, 0))`, I can return a value from column A even though I'm looking up in column E, which is to the right. This is something VLOOKUP simply cannot do.

2. Immunity to Column Insertions

The `col_index_num` in VLOOKUP is a hardcoded number. If someone inserts a column in the middle of the table, that number now points to the wrong column — silently giving wrong results without any error. INDEX-MATCH references the return column directly by range (like `D:D`), so inserting columns has zero effect on it.

3. Better Performance on Large Data

On very large datasets (100,000+ rows), VLOOKUP scans the entire `table_array`, while MATCH only scans the lookup column. INDEX-MATCH is noticeably faster in practice.

When I'd absolutely mandate it:

In any financial model maintained by a team over time — where people regularly add columns for new months, new metrics, or new scenarios. A shared model with VLOOKUP is a ticking time bomb. One column insertion and the entire model silently breaks. I'd make INDEX-MATCH a team standard in that context.

Q5. [Scenario — Descriptive]

Scenario: You receive a monthly customer invoice file. CustomerID in column A is stored as numbers (e.g., 1001, 1002). The CRM reference table has CustomerIDs stored as text (e.g., '1001', '1002'). Your VLOOKUP returns #N/A for every single row even though the IDs visually look identical. Walk through your complete diagnosis and fix process.

Answer:

Okay, this is actually a really common scenario I've encountered — it's the classic data type mismatch problem. Let me walk through exactly how I'd diagnose and fix it.

Step 1: Confirm the diagnosis

I'd click on one cell in column A and check the formula bar. If it's left-aligned, it's text. If it's right-aligned, it's a number. Then I'd check the CRM table the same way. I can also use `=ISNUMBER(A2)` on the lookup value and `=ISNUMBER(D2)` on the table's first column to get TRUE/FALSE confirmation.

Step 2: Understand why VLOOKUP fails here

VLOOKUP compares values with strict type checking. Even though 1001 and '1001' look identical on screen, Excel treats them as different types — a number vs a string. So the lookup fails silently with #N/A.

Step 3: The fix — align data types in the formula

If column A has numbers and the CRM table has text IDs, I convert in the formula:

```
=VLOOKUP(TEXT(A2,"0"), CustTable, 2, FALSE)
```

`TEXT(A2, "0")` converts the number 1001 to the text '1001' — now it matches the CRM table.

Alternatively, if the CRM table has numbers and column A has text:

```
=VLOOKUP(VALUE(A2), CustTable, 2, FALSE)
```

Step 4: The professional fix — standardize at source

I'd go one step further and fix the data type inconsistency at the source rather than just patching the formula. Using Power Query or a one-time paste-as-values with Text-to-Columns conversion, I'd ensure both tables use the same type. Formula patches are fragile — next month's file might have the same problem and a new analyst might not know to add `TEXT()`.

Pro tip: Prefer XLOOKUP for new work:

XLOOKUP has the same issue with data type mismatches, but the fix is identical — wrap with `TEXT()` or `VALUE()`. The real benefit is XLOOKUP's built-in `if_not_found` argument means I don't need an additional `IFERROR` wrapper while debugging.

03- SQL & Databases

SCENARIO-BASED DESCRIPTIVE QUESTIONS (4 Questions)

Q10 Scenario-Based — Revenue Report Corruption by NULLs

Scenario: Your company's weekly revenue dashboard is showing a total that is 15% lower than the finance team's Excel total. After investigation, you find that the `discount_amount` column has NULL values for about 20% of orders. The revenue formula is: `price * quantity - discount_amount`. How do you diagnose and fix this?

Answer

Ah, this is a classic NULL propagation bug — and honestly one of the most common silent errors in analytics dashboards. Let me walk through how I'd approach it.

First, the diagnosis. When `discount_amount` is NULL for a row, the expression `price * quantity - NULL` evaluates to NULL. NULL arithmetic propagates — one NULL in the formula makes the entire row's revenue NULL. `SUM()` then silently skips those NULL revenue rows. So roughly 20% of orders are contributing nothing to the total.

-- Step 1: Quantify the problem

```
SELECT COUNT(*) AS total_rows,  
       COUNT(discount_amount) AS non_null,  
       COUNT(*) - COUNT(discount_amount) AS null_count  
FROM orders;
```

Now the fix. The business logic matters here: does NULL discount mean 'no discount applied' (so I should treat it as 0) or does it mean 'discount data wasn't collected' (so I should flag those rows separately)?

If NULL discount means no discount, the fix is straightforward:

```
-- Fix: treat NULL discount as 0  
SELECT SUM(price * quantity - COALESCE(discount_amount, 0))  
       AS corrected_revenue  
FROM orders;
```

If it means 'unknown', I'd split the report: one total for orders with complete data, and a separate flag for orders where discount data is missing — so the finance team knows exactly what's included and what isn't. Never silently exclude rows from a revenue report.

DATABASE SCENARIO-BASED CODING (3 Questions)

Q18 DB Coding — Bank Loan Analysis with NULL Handling

Database: A bank's loan table has columns: loan_id, customer_id, loan_amount, interest_rate, approved_date, disbursed_date, status. Task: (1) Find all approved loans not yet disbursed (disbursed_date IS NULL), (2) Calculate average interest rate only for disbursed loans, (3) Show loan_id, loan_amount, COALESCE(disbursed_date, 'Pending Disbursement') as disbursement_status.

Answer

Good real-world banking scenario — let me tackle all three parts.

```
-- Part 1 & 3: Approved but not disbursed
```

```
SELECT
```

```
  loan_id,
```

```
 customer_id,
```

```
 loan_amount,
```

```
 interest_rate,
```

```
 approved_date,
```

```
 CAST(COALESCE(CAST(disbursed_date AS VARCHAR),
```

```
 'Pending Disbursement') AS VARCHAR)
```

```
 AS disbursement_status
```

```
FROM loan
```

```
WHERE status = 'Approved'
```

```
  AND disbursed_date IS NULL
```

```
ORDER BY approved_date ASC;
```

```
-- Part 2: Avg interest rate for disbursed only
```

```
SELECT
```

```
  ROUND(AVG(interest_rate), 2) AS avg_disbursed_rate,
```

```
  COUNT(*) AS disbursed_loan_count
```

```
FROM loan
```

```
WHERE disbursed_date IS NOT NULL;
```

Part 2 is clean because AVG() already ignores NULLs — but using WHERE disbursed_date IS NOT NULL makes the intent explicit and also ensures we're calculating the average only for the subset we care about, not all loans.

04- Python for Analytics

Section C: Coding Questions (4 Questions)

Q3: Write a Python script that takes a raw list of sales records (each a dict with 'region', 'sales', 'month') and uses string manipulation, comparison operators, and logical operators to filter records where sales > 50000 AND region is either 'North' or 'South', then prints an f-string summary for each.

```
sales_records = [
    {'region': ' North ', 'sales': 75000, 'month': 'January'},
    {'region': 'South', 'sales': 43000, 'month': 'February'},
    {'region': 'East', 'sales': 90000, 'month': 'March'},
    {'region': 'North', 'sales': 62000, 'month': 'April'},
    {'region': 'South', 'sales': 51000, 'month': 'May'},
```

Answer:]

```
target_regions = ['North', 'South']
```

```
filtered = [
    r for r in sales_records
    if r['region'].strip() in target_regions # clean + membership check
    and r['sales'] > 50000 # comparison filter
```

Answer:]

```
for r in filtered:
    region = r['region'].strip()
    print(f"[{r['month']}] {region} - Sales: ₹{r['sales']:,.}")
```

```
# Output:
# [January] North - Sales: ₹75,000
# [April] North - Sales: ₹62,000
# [May] South - Sales: ₹51,000
```

```
# Key technique: r['region'].strip() cleans before comparing -
# without this, ' North ' != 'North' and the record is dropped silently.
```

Section D: Database Scenario-Based Coding (3 Questions)

These questions simulate working with database-like structured data (list of dicts) using Python fundamentals — the precursor to pandas and SQL queries.

Q1: Database Scenario: You have a transactions table (list of dicts) with fields: transaction_id (int), customer_name (str), amount (float), status (str), region (str). Write Python code to: (1) filter all 'Completed' transactions from the 'West' region with amount > 10000, and (2) calculate the total amount for those records.

```
transactions = [
    {'transaction_id': 101, 'customer_name': 'Arjun Nair', 'amount': 15000.0,
     'status': 'Completed', 'region': 'West'},
    {'transaction_id': 102, 'customer_name': 'Meena Pillai', 'amount': 8000.0,
     'status': 'Pending', 'region': 'West'},
    {'transaction_id': 103, 'customer_name': 'Suresh Rao', 'amount': 22000.0,
     'status': 'Completed', 'region': 'West'},
    {'transaction_id': 104, 'customer_name': 'Kavya Joshi', 'amount': 12000.0,
     'status': 'Completed', 'region': 'East'},
    {'transaction_id': 105, 'customer_name': 'Nitin Desai', 'amount': 30000.0,
     'status': 'Completed', 'region': 'West'},
```

Answer:]

```
# Step 1: Filter — equivalent to SQL WHERE clause
filtered = [
    t for t in transactions
    if t['status'] == 'Completed'
    and t['region'] == 'West'
    and t['amount'] > 10000
```

Answer:]

```
# Step 2: Aggregate — equivalent to SQL SUM()
total = sum(t['amount'] for t in filtered)

print(f'Matching records: {len(filtered)}')
for t in filtered:
    print(f" ID {t['transaction_id']} | {t['customer_name']} | ₹{t['amount']:,.2f}")
print(f'Total Amount: ₹{total:,.2f}')

# Output:
# Matching records: 2
# ID 101 | Arjun Nair | ₹15,000.00
# ID 103 | Suresh Rao | ₹22,000.00 <- Wait, let me recheck...
# Actually ID 103 matches, ID 105 matches too (30000 > 10000)
# Total Amount: ₹67,000.00

# SQL equivalent:
# SELECT *, SUM(amount) FROM transactions
# WHERE status = 'Completed' AND region = 'West' AND amount > 10000
```

05- Business Intelligence & Dashboards

Section C: Coding Questions (4 Questions)

Q1. Using Python and pandas, write code to calculate both a Lead KPI (7-day rolling average of daily active users) and a Lag KPI (monthly total revenue). The dataset has columns: date, daily_active_users, and revenue.

Answer

Understanding the "why" here matters: rolling averages smooth out daily noise so you can see the trend signal clearly — that's what makes it useful as a lead indicator. Monthly total is a clean lag confirmation.

Code:

```
import pandas as pd

# Sample dataset
data = {
    "date": pd.date_range(start="2024-01-01", periods=90, freq="D"),
    "daily_active_users": [200 + i*2 for i in range(90)],
    "revenue": [5000 + i*100 for i in range(90)]
}
df = pd.DataFrame(data)
df["date"] = pd.to_datetime(df["date"])

# — LEAD INDICATOR: 7-Day Rolling Average of Daily Active Users —
df["rolling_avg_dau"] = df["daily_active_users"].rolling(window=7).mean()

# — LAG INDICATOR: Monthly Total Revenue —
df["month"] = df["date"].dt.to_period("M")
monthly_revenue = df.groupby("month")["revenue"].sum().reset_index()
monthly_revenue.columns = ["month", "total_monthly_revenue"]

# — Output —
print("Lead Indicator - 7-Day Rolling Avg DAU:")
print(df[["date", "daily_active_users", "rolling_avg_dau"]].tail(10))

print("\nLag Indicator - Monthly Total Revenue:")
print(monthly_revenue)
```

Section D: Database Scenario-Based Questions (3 Questions)

Q1. DATABASE SCENARIO: You have a `sales_transactions` table with columns: `transaction_id`, `salesperson_id`, `region`, `sale_date`, `revenue`, and `target_revenue`. Write SQL to generate a KPI summary report showing: total actual revenue, total target revenue, attainment %, and RAG status (Green $\geq 100\%$, Amber $\geq 90\%$, Red $< 90\%$) — grouped by region. Sort by attainment ascending (worst-performing regions first).

Answer

This is a classic dashboard backend query — grouping real vs. target data and applying business logic (RAG classification) inside SQL. Sorting worst-first is intentional: it ensures problem regions are visible immediately on the dashboard.

SQL:

```
SELECT
  region,
  SUM(revenue) AS total_actual_revenue,
  SUM(target_revenue) AS total_target_revenue,
  ROUND(
    SUM(revenue) * 100.0 / NULLIF(SUM(target_revenue), 0),
    2) AS attainment_pct,
  CASE
    WHEN SUM(revenue) >= SUM(target_revenue)
    THEN 'GREEN - On/Above Target'
    WHEN SUM(revenue) >= SUM(target_revenue) * 0.90
    THEN 'AMBER - Within 10% of Target'
    ELSE
    'RED - Below 90% of Target'
  END AS rag_status
FROM
  sales_transactions
GROUP BY
  region
ORDER BY
  attainment_pct ASC; -- Worst performers first (critical for dashboard design)

-- NULLIF prevents division-by-zero when target is 0
-- Multiply by 100.0 (not 100) to force decimal division in integer DBs
```

06- Data Engineering Fundamentals

Coding — Q1

Write a Python script to call the Zendesk REST API, retrieve all open tickets with pagination, and store them in a list. Handle authentication with a Bearer token.



Answer:

The key things I'm keeping in mind here: auth header, handling the next_page loop for pagination, and collecting all tickets into a single list.

```
import requests

BASE_URL = "https://yourcompany.zendesk.com/api/v2"
API_TOKEN = "YOUR_ZENDESK_API_TOKEN"

headers = {
    "Authorization": f"Bearer {API_TOKEN}",
    "Content-Type": "application/json"
}

def fetch_all_open_tickets():
    tickets = []
    url = f"{BASE_URL}/tickets.json?status=open"

    while url:
        response = requests.get(url, headers=headers)

        if response.status_code == 429: # Rate limit hit
            print("Rate limited - waiting 60s")
            import time; time.sleep(60)
            continue

        response.raise_for_status() # Raise for 4xx/5xx errors
        data = response.json()
        tickets.extend(data.get("tickets", []))
        url = data.get("next_page") # None stops the loop

    return tickets

all_tickets = fetch_all_open_tickets()
print(f"Total tickets fetched: {len(all_tickets)}")
```

Why I structured it this way: The while url: loop keeps fetching as long as next_page is not None. I added a 429 handler because rate limiting is a real-world problem with Zendesk. Using raise_for_status() ensures we catch errors early rather than silently processing empty data.

Database — Q1

Scenario: You have a staging table `staging.orders_cdc` that contains CDC events (INSERT, UPDATE, DELETE) from your source system. Write a Snowflake SQL MERGE statement to apply these changes to your analytics.orders target table. Ensure that new records are inserted, existing records are updated, and deleted records are removed.

Answer:

This is the classic CDC merge pattern — the foundation of any incremental pipeline. Here's how I'd approach it in Snowflake:

```
-- -----
-- CDC MERGE: Apply INSERT / UPDATE / DELETE from staging
-- -----

MERGE INTO analytics.orders AS target
USING (
  -- De-duplicate: keep only the latest CDC event per order_id
  SELECT *
  FROM (
    SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY order_id
      ORDER BY event_timestamp DESC
    ) AS rn
    FROM staging.orders_cdc
    WHERE DATE(event_timestamp) = CURRENT_DATE
  )
  WHERE rn = 1
) AS source
ON target.order_id = source.order_id

-- Update existing records (INSERT or UPDATE events)
WHEN MATCHED AND source.operation IN ('INSERT', 'UPDATE') THEN
  UPDATE SET
    target.customer_id = source.customer_id,
    target.order_status = source.order_status,
    target.order_amount = source.order_amount,
    target.updated_at = source.event_timestamp

-- Remove deleted records
WHEN MATCHED AND source.operation = 'DELETE' THEN
  DELETE

-- Insert brand-new records
WHEN NOT MATCHED AND source.operation != 'DELETE' THEN
  INSERT (order_id, customer_id, order_status, order_amount,
    updated_at)
  VALUES (source.order_id, source.customer_id,
```

```
source.order_status, source.order_amount,  
source.event_timestamp);
```

Why the de-duplication subquery? If the staging table gets multiple CDC events for the same `order_id` (e.g., an UPDATE followed immediately by another UPDATE), we only want to apply the **latest** one. The `ROW_NUMBER()` window function handles this cleanly. Without it, the MERGE might fail on duplicate source rows.

07- Storytelling, Communication & Domain Knowledge

SECTION B — Scenario-Based Descriptive Questions (4 Questions)

Q1 [Scenario] Scenario: The Head of Marketing at an e-commerce company tells you: 'Our email campaigns aren't working. Can you look into it?' Walk through how you would apply the 5 Whys technique and then convert the finding into a precise data question.

My Approach:

The first thing I'd recognize is that 'email campaigns aren't working' is a symptom, not a root cause. Before I open any reporting tool, I'd sit with the Head of Marketing and run a 5 Whys chain.

5 Whys Chain:

Why 1: Why aren't email campaigns working? → Open rates are very low, around 12%.

Why 2: Why are open rates low at 12%? → Subject lines are generic and not personalized.

Why 3: Why are subject lines not personalized? → We're sending the same email to all 200,000 subscribers.

Why 4: Why are we sending to everyone? → We don't have audience segmentation in place.

Why 5: Why don't we have segmentation? → Customer purchase behavior hasn't been tagged in the CRM.

Root Cause:

The root cause is lack of CRM customer segmentation, not a creative or content problem as the stakeholder assumed.

Converted Data Question:

'What is the open rate, click-through rate, and conversion rate of weekly product launch emails for each customer purchase behavior segment (first-time buyers, repeat buyers, lapsed buyers) over the last 3 months?' — This tells us exactly which segments to target and what messaging is resonating.

SECTION D — Database Scenario-Based Questions (3 Questions)

Q1 [DB Coding] Database Scenario: You have a table 'orders' with columns: order_id, customer_id, order_date, total_amount, product_category, region, status. Write a SQL query to identify the top 3 product categories by return rate (status = 'Returned') for the last 6 months, along with total returns and total orders per category.

Understanding the Business Need:

This is a direct application of Topic 1.2 — converting a business problem ('returns are rising') into a data question ('Which product categories have the highest return rate in the last 6 months?'). The SQL needs to aggregate by category, compute both counts, and calculate the rate.

SQL Query:

```
-- Top 3 Product Categories by Return Rate (Last 6 Months)
SELECT
  product_category,
  COUNT(*) AS total_orders,
  SUM(CASE WHEN status = 'Returned' THEN 1 ELSE 0 END)
  AS total_returns,
  ROUND(
    SUM(CASE WHEN status = 'Returned' THEN 1 ELSE 0 END) * 100.0
    / COUNT(*),
    2
  ) AS return_rate_pct
FROM
  orders
WHERE
  order_date >= CURRENT_DATE - INTERVAL '6 months' -- Last 6 months filter
  AND order_date < CURRENT_DATE
GROUP BY
  product_category
HAVING
  COUNT(*) >= 50 -- Exclude categories with too few orders (statistical
  noise)
ORDER BY
  return_rate_pct DESC
LIMIT 3;
```

Query Design Handbook:

The HAVING COUNT(*) >= 50 guard is important — without it, a category with 2 orders and 1 return would show as 50% return rate, which is statistically meaningless. The ROUND(..., 2) keeps percentages clean for reporting. I'd also add a CTE version if the stakeholder wants to filter by region as well.

08- AI-Era Analytics

Section B — Scenario-Based Descriptive Questions

Q5 [Scenario-Based]

Scenario: You are a junior analyst at a healthcare company. Your manager asks you to use AI to analyse patient readmission rates. The AI keeps giving generic statistical responses that do not align with your clinical database structure or the team's goals. What specific Analytical Persona prompt would you design? Write out the full persona prompt and explain each component you included.

Answer

This scenario is a classic case where a persona is missing — the AI doesn't know it's working in healthcare, doesn't know the tools, and doesn't know the goal.

My designed persona prompt:

"You are a Senior Clinical Data Analyst with 8 years of experience in hospital operations and patient outcomes analytics.

You work at a large multi-specialty hospital that uses SQL Server and Python (pandas, matplotlib) for analysis.

Your goal is to help the clinical team reduce 30-day patient readmission rates by identifying high-risk patient segments.

You always reference ICD-10 diagnosis codes correctly, validate date ranges for admission and discharge, and explain your SQL logic.

You never use `SELECT *` and always filter for completed, non-test patient records.

Now answer the following question: [ACTUAL QUESTION]"

Why each component matters:

- Role + Experience: Anchors the AI in clinical analytics, not generic stats
- Tools: Ensures SQL is written for SQL Server syntax, not MySQL or PostgreSQL
- Goal: The AI now prioritises readmission-relevant metrics (e.g., LOS, diagnosis codes)
- Rules: Prevents ICD code hallucinations and ensures data hygiene
- Restrictions: Avoids lazy `SELECT *` queries that could expose PHI unnecessarily

Section D — Database Scenario-Based Coding (SQL)

Q13 [DB Scenario + SQL]

Scenario: You are a data analyst at a SaaS company. Using the schema below, write an AI-ready Few-Shot prompt that includes the schema, one Gold Standard example, and then solve the new question yourself with a complete SQL query. Schema: Table: subscriptions (sub_id INT, customer_id INT, plan VARCHAR, start_date DATE, end_date DATE, monthly_fee FLOAT, status VARCHAR) Table: customers (customer_id INT, full_name VARCHAR, country VARCHAR, signup_date DATE) New Question: Find the total Monthly Recurring Revenue (MRR) by plan type for all active subscriptions.

Answer

I'll structure this as both the Few-Shot prompt (what you'd send to the AI) AND the direct SQL solution — since the question asks for both.

The Few-Shot prompt to send to AI:

Schema: subscriptions + customers (as above)

Gold Standard Example: Count active subscriptions per country

Direct SQL Solution:

Code:

```
-- Few-Shot Gold Standard Example (included in prompt):
-- Q: Count active subscriptions per country.
SELECT c.country,
       COUNT(s.sub_id) AS active_subscriptions
FROM subscriptions s
JOIN customers c ON s.customer_id = c.customer_id
WHERE s.status = 'active'
GROUP BY c.country
ORDER BY active_subscriptions DESC;

-- — SOLUTION: MRR by Plan Type —————
SELECT
  plan,
  COUNT(sub_id) AS active_subscriptions,
  SUM(monthly_fee) AS total_mrr,
  ROUND(AVG(monthly_fee), 2) AS avg_fee_per_plan
FROM subscriptions
WHERE status = 'active'
GROUP BY plan
ORDER BY total_mrr DESC;

-- Why this is correct:
-- • Filters status = 'active' to exclude churned/cancelled subs
-- • SUM(monthly_fee) gives true MRR per plan
```

```
-- • AVG gives insight into plan pricing distribution
-- • ORDER BY total_mrr DESC shows highest-revenue plans first
```